Gestion des événements universitaires

Équipe Warriors

Eya Ben Saber · Eya Baccouche · Rayen Amri · Youssef Saibi · Hana Abidi · Amal Barhoumi

Scannez : dépôt Git

Sommaire

01. Problème et périmètre

02. Acteurs et fonctionnalités

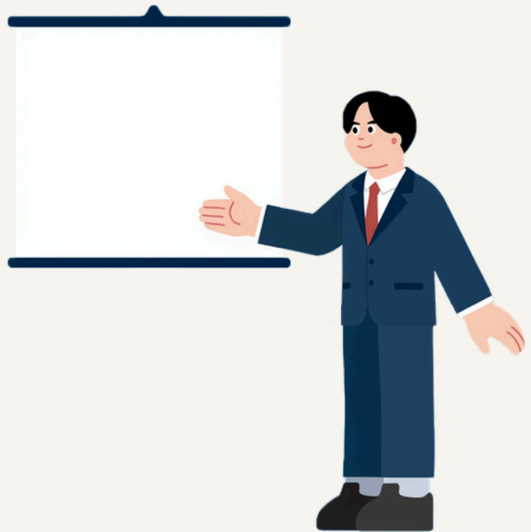
03. Découpage en sept microservices

04. Conception métier des microservices

05. Conception du microservice commun

06. Communications synchrones et asynchrones

07. Architecture et technologies



Problème et périmètre

Le constat

- Inscriptions suivies dans des fichiers dispersés
- Salles réservées à la main : risque de double réservation
- Matériel et sponsors gérés par échanges informels
- Aucune aide pour former une équipe, aucune récompense de l'engagement

Notre réponse

Une plateforme web unique pour gérer tout le cycle d'un événement universitaire : inscription, réservation de salle, sponsoring, compétitions et équipes, logistique, avis et fidélité. Elle repose sur sept microservices : six individuels, un par membre de l'équipe, et un microservice commun, Logistique & Équipements, développé en Python.

Hors périmètre : paiement en ligne, diffusion vidéo et gestion des comptes, confiée à Keycloak (il n'y a donc pas de microservice utilisateur).

Acteurs et fonctionnalités

Participant (étudiant)

PARTICIPANT

- Consulter les événements et s’inscrire
- Créer ou rejoindre une équipe
- Demander le Smart Matching solo
- Déposer le projet / livrable
- Consulter et convertir ses points de fidélité
- Donner un avis et évaluer

Organisateur (club / BDE)

ORGANISATEUR

- Gérer les événements : créer, publier, modifier
- Réserver une salle ou un amphithéâtre
- Affecter intervenants et jurys
- Emprunter du matériel logistique (kits IoT, micros, câbles)
- Solliciter un sponsor pour un événement

Intervenant / Jury

INTERVENANT

- Déclarer ses disponibilités
- Consulter ses affectations (speaker / jury)
- Évaluer les projets des équipes

Sponsor / Partenaire

SPONSOR

- Consulter les packs de sponsoring
- Traiter une sollicitation (accepter / refuser)
- Souscrire une convention de partenariat

Administrateur

ADMIN

- Gérer le référentiel des salles
- Gérer le stock d’équipements logistiques
- Superviser les événements et conventions

Découpage en sept microservices

Événements
Cycle de vie des événements et inscriptions avec contrôle de capacité

Eya Ben Saber — MySQL

Compétitions & Équipes
Équipes, dépôt de projets et Smart Matching des étudiants solos

Eya Baccouche — PostgreSQL

Sponsors & Partenariats
Packs de sponsoring, sollicitations et conventions de partenariat

Rayen Amri — MySQL

Feedback et intervenants
Avis des participants, profils et affectations des intervenants

Youssef Saibi — PostgreSQL

Fidélité & Récompenses
Points, badges et récompenses ; abonné aux événements

Hana Abidi — MongoDB

Planning des salles
Salles, réservations et détection des conflits de créneau

Amal Barhoumi — PostgreSQL

Logistique & Équipements
Inventaire du matériel, kits IoT et stands : emprunts, retours et état du parc. Microservice commun développé en Python, port 8087.

Toute l'équipe — Python, PostgreSQL

MICROSERVICE 1 SUR 7 · EYA BEN SABER

Événements

Crée et publie les événements, gère les inscriptions et contrôle la capacité. C’est le service pivot : tous les appels synchrones passent par lui.

- Suit la vie d’un événement : brouillon, publié, clôturé ou annulé.
- Inscrit les participants tant qu’il reste de la place.
- Prévient les autres services à la création et à chaque inscription.

API REST

Méthode	URI	Codes
POST	/events	201 · 400
GET	/events?type=&statut=	200
PUT	/events/{id}/publish	200 · 409
POST	/events/{id}/registrations	201 · 409
GET	/api/evenements/{id}/statut	200 · 404
DELETE	/registrations/{id}	204 · 404

Données

MySQL

Événement : id, titre, description, type, dateDebut, dateFin, salleId, capacité, statut

Inscription : id, evenementId (ManyToOne), participantId, dateInscription, statut

Pourquoi MySQL : données relationnelles et contrôle transactionnel de la capacité.

Interactions

SYNC Appelle Planning pour vérifier la disponibilité de la salle avant de publier : un conflit bloque la publication. Appelé par Sponsors, Compétitions, Feedback et Logistique.

ASYNC Publie evenement.cree et inscription.validee. Consomme materiel.restitue (clôture du volet matériel).

PANNE Si Planning est arrêté, l’événement reste en brouillon.

MICROSERVICE 2 SUR 7 · EYA BACCOUCHE

Compétitions & Équipes

Organise les hackathons : équipes, dépôt de projets et Smart Matching pour les étudiants solos sans équipe.

- Crée les équipes ou fait rejoindre par code d'invitation.
- Propose une équipe aux étudiants solos (Smart Matching).
- Reçoit le dépôt du projet ; le jury attribue la note finale.

Données

PostgreSQL

Equipe : id, evenementId, nomEquipe, codeInvitation, tableId, dateCreation

MembreEquipe : id, equipeId, participantId, roleCompetence, estChefEquipe

ProjetSoumis : id, equipeId, titre, repoUrl, dateDepot, noteFinale

Pourquoi PostgreSQL : requêtes relationnelles (équipes, membres, projets) et calcul du Smart Matching en SQL.

API REST

Méthode	URI	Codes
POST	/competitions/teams	201 · 403 · 409
POST	/competitions/teams/join	200 · 404 · 409
POST	/competitions/matching	200 · 404
POST	/competitions/teams/{id}/projects	201 · 409
PUT	/competitions/projects/{id}/grade	200 · 403 · 404

Interactions

SYNC Appelle Événements pour vérifier que l'étudiant est inscrit avant d'intégrer une équipe (GET /api/evenements/{id}/inscriptions/{etudiantId}).

ASYNC Publie equipe.formee. Ne consomme aucun message.

PANNE Si Événements est arrêté, la création ou l'adhésion est refusée, puis nouvel essai.

MICROSERVICE 3 SUR 7 · RAYEN AMRI

Sponsors & Partenariats

Gère les packs de sponsoring, les conventions et le démarchage entre organisateurs et sponsors.

- Définit les packs de sponsoring : tarif, stand inclus.
- L'organisateur sollicite un sponsor ; il accepte ou refuse.
- Une sollicitation acceptée donne une convention de partenariat.

Données

MySQL

Sponsor : id, nomEntreprise, contactEmail, logoUrl

PackSponsoring : id, nomPack, tarif, standInclus

ConventionPartenariat : id, sponsorId, packId, evenementId, budgetAlloue, statutPaiement

Pourquoi MySQL : contraintes d'intégrité et transactions sur les conventions.

API REST

Méthode	URI	Codes
POST	/sponsors/packs	201 · 403
GET	/sponsors/packs	200
POST	/sponsors/solicitations	201 · 404 · 409
PUT	/sponsors/solicitations/{id}/accept	200 · 404 · 409
PUT	/sponsors/solicitations/{id}/reject	200 · 404 · 409
POST	/sponsors/conventions	201 · 409

Interactions

SYNC — Appelle Événements : l'événement doit exister et être au statut PUBLIÉ (GET /api/evenements/{id}).

ASYNC — Publie convention.validee. Consomme materiel.restitue (libération des cautions).

PANNE — Si Événements est arrêté, la sollicitation est refusée, puis nouvel essai.

MICROSERVICE 4 SUR 7 · YOUSSEF SAIBI

Feedback et intervenants

Recueille les avis des participants et gère les intervenants, jurys et leurs affectations.

- Recueille les avis des participants après l'événement.
- Gère les profils des intervenants et des jurys.
- Les affecte à un événement comme conférencier ou jury.

Données

PostgreSQL

Avis : id, evenementId, participantId, note, commentaire, dateAvis

Intervenant : id, nom, fonction, disponibilites

Affectation : id, intervenantId, evenementId, rôle

Pourquoi PostgreSQL : base relationnelle robuste, commune avec Planning, Compétitions et Logistique.

API REST

Méthode	URI	Codes
POST	/events/{id}/feedback	201 · 403 · 409
GET	/events/{id}/feedback	200
POST	/speakers	201
POST	/assignments	201 · 409
GET	/speakers/{id}/assignments	200

Interactions

SYNC Appelle Événements : l'événement doit être CLÔTURÉ avant le dépôt d'un avis (GET /api/evenements/{id}/statut).

ASYNC Publie avis.publie. Ne consomme aucun message.

PANNE Si Événements est arrêté, l'avis est refusé et peut être réessayé.

MICROSERVICE 5 SUR 7 · HANA ABIDI

Fidélité & Récompenses

Anime la gamification : points d'engagement, badges et récompenses. Il est uniquement abonné aux événements des autres services.

- Crédite des points d'engagement à chaque action.
- Attribue des badges et un niveau de fidélité.
- Échange les points contre des récompenses.

Données

MongoDB

CompteFidelite : id, participantId, soldePoints, niveau, badges, dateDerniereActivite

TransactionPoints : id, participantId, evenementId, typeAction, points, date

Recompense : id, titre, coutPoints, stockRestant, actif

Pourquoi MongoDB : documents imbriqués (historique de points) et écritures fréquentes.

API REST

Méthode	URI	Codes
GET	/loyalty/account/me	200
GET	/loyalty/transactions/me	200
GET	/loyalty/rewards	200
POST	/loyalty/rewards	201 · 403
POST	/loyalty/rewards/{id}/redeem	200 · 409

Interactions

SYNC Aucun appel vers les autres services.

ASYNC Consomme inscription.validee (+50 points), equipe.formee (badge Team Player, +100 points) et avis.publie (+20 points).

PANNE Si Fidélité est arrêté, les messages attendent dans le broker et les points sont crédités à son retour.

MICROSERVICE 6 SUR 7 · AMAL BARHOUMI

Planning des salles

Gère les salles et empêche les réservations qui se chevauchent.

- Gère les salles et leurs réservations.
- Refuse tout chevauchement de créneau.
- Verrouille la salle dès la création de l'événement.

Données

PostgreSQL

Salle : id, nom, batiment, capacite, typeSalle

Reservation : id, salleId, evenementId, debut, fin, statut

Pourquoi PostgreSQL : une contrainte d'exclusion sur les plages horaires interdit les conflits.

API REST

Méthode	URI	Codes
GET	/rooms?minCapacity=	200
POST	/rooms	201 · 403
POST	/reservations	201 · 409 · 422
DELETE	/reservations/{id}	204 · 404
GET	/api/salles/{id}/disponibilite	200 · 404

Interactions

SYNC — Appelé uniquement par Événements (disponibilité de la salle sur un intervalle). Planning n'appelle jamais Événements.

ASYNC — Consomme evenement.cree : pose un verrou sur la salle retenue et bloque les créneaux concurrents.

MICROSERVICE COMMUN · TOUTE L'ÉQUIPE

Logistique & Équipements

Gère le matériel, les kits IoT et les stands. Développé en commun, en Python, sur le port 8087.

- Tient l'inventaire du matériel et des kits IoT.
- Traite les demandes d'emprunt des organisateurs.
- Suit les retours et l'état du matériel.

Données

Python PostgreSQL

Equipement : id, designation, categorie, quantiteStock, quantiteDisponible

DemandeEmprunt : id, equipementId, evenementId, demandeurId, quantite, dateEmprunt, dateRetourPrevue, statut

Pourquoi PostgreSQL : transactions et contraintes sur les stocks, comme les autres services relationnels.

API REST

Méthode	URI	Codes
GET	/logistics/equipment?category=	200
POST	/logistics/equipment	201 · 403
POST	/logistics/loans	201 · 404 · 409
PUT	/logistics/loans/{id}/return	200 · 404
GET	/logistics/loans?eventId=	200

Interactions

SYNC Appelle Événements : la demande doit être rattachée à un événement vérifié (GET /api/evenements/{id}).

ASYNC Consomme evenement.cree (pack matériel standard), equipe.formee (kits IoT) et convention.validee (stand sponsor). Publie materiel.restitue.

PANNE Si Événements est arrêté, la demande d'emprunt est refusée, puis nouvel essai.

SOCLE COMMUN

Gateway & Infrastructure

Point d'entrée unique, sécurité JWT, découverte des services et configuration centralisée.

Composants

Spring Cloud

API Gateway (8080) : point d’entrée unique du front-end React (5173).

Eureka Server (8761) : annuaire dynamique des 7 microservices, service Python compris.

Config Server (8888) : centralisation des fichiers de configuration.

Routage Gateway

Préfixe URI	Service cible	Route Eureka
/events/**	Événements	lb://evenements
/rooms/**, /reservations/**	Planning	lb://planning
/feedback/**, /speakers/**	Feedback	lb://feedback
/competitions/**	Compétitions	lb://competitions
/sponsors/**	Sponsors	lb://sponsors
/loyalty/**	Fidélité	lb://fidelite
/logistics/**	Logistique (Python)	lb://logistique

Interactions

SÉCURITÉ Keycloak (8180) gère comptes, rôles et jetons JWT ; la Gateway vérifie le jeton avant de router.

DOCKER Docker Compose orchestre 7 microservices, bases, RabbitMQ, Keycloak et le socle Spring Cloud.

Communication synchrone (OpenFeign) · 1/2

Règle : synchrone quand l’appelant a besoin de la réponse tout de suite. Cinq appels REST, tous vers ou depuis Événements.

Événements → Planning

GET /api/salles/{id}/disponibilite?debut=&fin=

Règle bloquante

Salle libre sur l’intervalle, sinon la publication est bloquée.

Si l’appelé est arrêté

Si Planning est arrêté, l’événement reste en brouillon.

Sponsors → Événements

GET /api/evenements/{id}

Règle bloquante

L’événement existe et est au statut PUBLIÉ avant une convention.

Si l’appelé est arrêté

Refus, puis nouvel essai.

Compétitions → Événements

GET /api/evenements/{id}/inscriptions/{etudiantId}

Règle bloquante

L’étudiant est inscrit avant d’intégrer une équipe.

Si l’appelé est arrêté

Refus, puis nouvel essai.

Communication synchrone (OpenFeign) · 2/2

Les services de Compétitions, Feedback et Logistique appellent Événements, qui reste le service pivot.

Feedback → Événements

GET /api/evenements/{id}/statut

Règle bloquante

L'événement est CLÔTURÉ avant le dépôt d'un avis.

Si l'appelé est arrêté

Refus, puis nouvel essai.

Logistique → Événements

GET /api/evenements/{id}

Règle bloquante

Le matériel est rattaché à un événement vérifié.

Si l'appelé est arrêté

Refus, puis nouvel essai.

Communication asynchrone (RabbitMQ) · 1/2

evenement.cree
Événements → Planning, Logistique

Traitement

Planning verrouille la salle et bloque les créneaux concurrents ; Logistique pré-réserve le pack matériel standard.

Si le service est arrêté

Le message reste dans le broker et est traité au retour du service.

inscription.validee
Événements → Fidélité

Traitement

Fidélité ajoute +50 points au compte du participant, sans bloquer l’inscription.

Si le service est arrêté

Le message reste dans le broker et est traité au retour du service.

equipe.formee
Compétitions → Fidélité, Logistique

Traitement

Fidélité attribue le badge Team Player et +100 points ; Logistique prépare les kits IoT et multiprises.

Si le service est arrêté

Le message reste dans le broker et est traité au retour du service.

Communication asynchrone (RabbitMQ) · 2/2

avis.publie
Feedback → Fidélité

Traitement
Fidélité crédite +20 points pour l'avis donné.

Si le service est arrêté
Le message reste dans le broker et est traité au retour du service.

convention.validee
Sponsors → Logistique

Traitement
Logistique attribue le stand, les roll-ups et les écrans dès la validation de la convention.

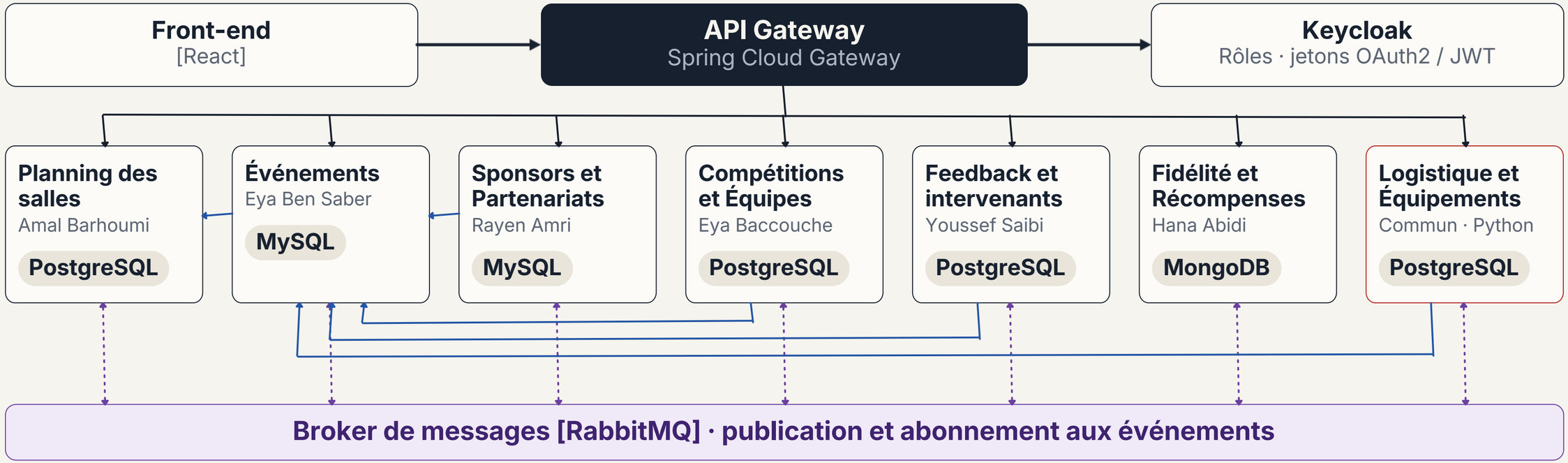
Si le service est arrêté
Le message reste dans le broker et est traité au retour du service.

materiel.restitue
Logistique → Événements, Sponsors

Traitement
Événements clôture le volet matériel ; Sponsors libère les cautions et conventions.

Si le service est arrêté
Le message reste dans le broker et est traité au retour du service.

Architecture technique



INFRASTRUCTURE SPRING CLOUD ET CONTENEURS

Eureka Server
Annuaire des 7 services

Config Server
Configuration centralisée

Docker Compose
Un conteneur par service

Appel REST synchrone (OpenFeign) · Événement asynchrone · Routage de la Gateway · Service commun

Merci de votre attention

Gestion des événements universitaires · Équipe Warriors

· Eya Ben Saber · Eya Baccouche · Rayen Amri · Youssef Saibi · Hana Abidi · Amal Barhoumi

